NestSelf authentication and authorization: implementation guide

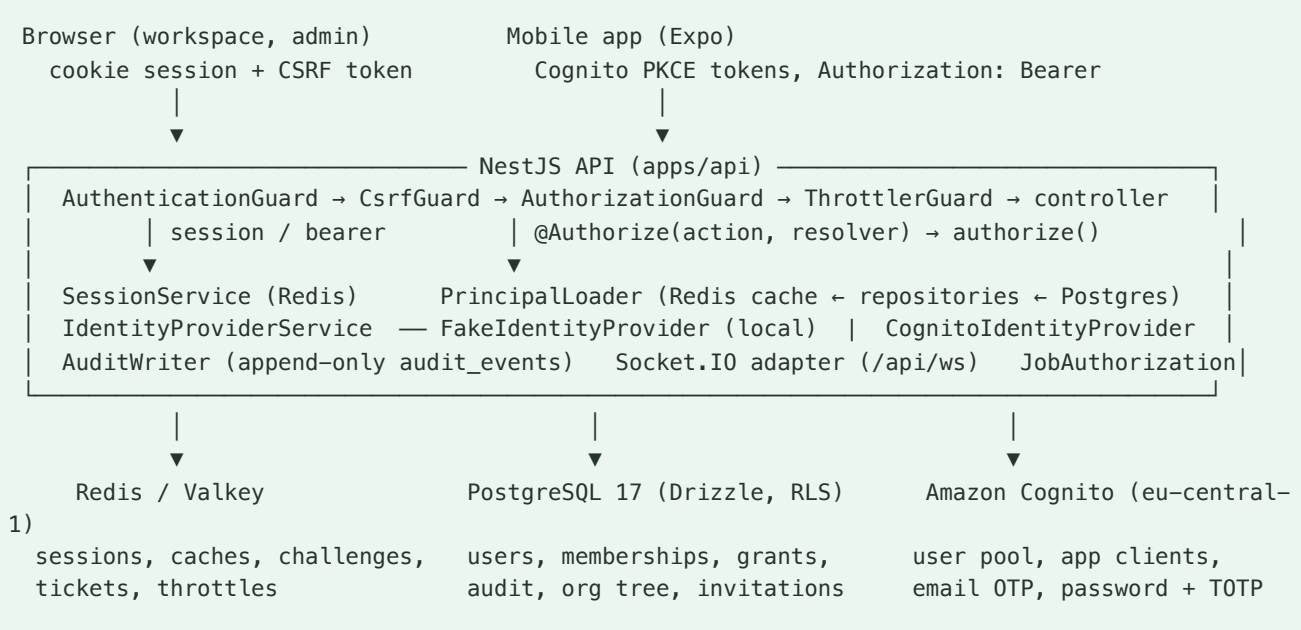
How identity, sessions, authorization, the data layer and the clients fit together, where each piece lives in the repository, and how to work on it. The companion `quick-guide.md` gives the one-page version and the reasoning; `docs/tasks/auth/auth-strategy.md` is the full specification (requirement ids such as `D-06` or `AUTHZ-R-61` below refer to it).

Last updated 2026-10-23 on branch `feat/auth-foundation`.

Contents

1. The system in one picture
2. Running it locally
3. Repository map
4. The request pipeline
5. Identity provider layer
6. Sign-in flows
7. Sessions, cookies and CSRF
8. Mobile bearer tokens
9. Principal, grants and the authorization engine
10. Data layer: Drizzle, repositories and row-level security
11. Organization and operations administration
12. Audit log
13. WebSockets
14. Background jobs
15. Web clients
16. Mobile client
17. Infrastructure and the Cognito rollout
18. Configuration reference
19. Testing
20. Recipes
21. Known gaps and next steps

1. The system in one picture



Three ideas carry the whole design:

- **Identity is not authorization.** Cognito (or the local fake) only proves who the caller is. What they may do comes from `grants`, `assignments` and `memberships` in PostgreSQL, resolved per request into a `Principal` and evaluated by the pure `authorize()` function. Tokens carry no roles (D-02).
- **Browsers never see tokens.** Web clients hold an opaque session cookie; the API keeps the Cognito tokens encrypted in Redis (Backend-for-Frontend, D-06). Mobile holds Cognito tokens itself and calls the API with a bearer token (D-07).
- **Default deny, twice.** Every route must declare `@Public()`, `@Authenticated()` or `@Authorize()`, enforced by a CI test. Tenant tables also carry PostgreSQL row-level security policies, so a query that forgets its organization filter returns nothing (D-19, D-37).

2. Running it locally

Local development uses `AUTH_MODE=stub`: a fake identity provider in memory, an in-memory key-value store unless `REDIS_URL` is set, and a real PostgreSQL.

```
pnpm install
pnpm db:up                                  # Postgres 17 and Redis 7 via Docker Compose
cp apps/api/.env.example apps/api/.env      # defaults are fine for stub mode
pnpm --filter @nestself/api db:migrate      # apply apps/api/drizzle/*.sql
pnpm --filter @nestself/api db:seed         # fictional fixtures from packages/sample-data
pnpm --filter @nestself/api dev             # http://localhost:4000, Swagger at /api/docs
pnpm --filter @nestself/api worker:dev      # optional: the nightly jobs (needs REDIS_URL)
pnpm --filter @nestself/api job:run grants.reconcile   # run one job once, no queue
```

The seed creates the sample organization, org units, people, grants and an `external_identities` row per person. The fake identity provider is pre-loaded with the same people (`apps/api/src/identity/identity.module.ts`), so sign-in works out of the box:

Sample person	Email	Population	How to sign in
Jovan Petrović	`user_jovan@example.com`	employee	Email code; read it from the API log line `Fake identity provider: OTP for ... is 123456`, or set `FAKE_IDP_FIXED_OTP=123456`
Ivana Jovanović	`user_ivana@example.com`	employer HR admin	Email code
Stefan Marković	`user_stefan@example.com`	team manager	Email code
Nikola Đorđević	`user_nikola@example.com`	operations staff	Password `Nestself-dev-1234!`, then authenticator enrolment on first sign-in
Milica Ristić	`user_milica@example.com`	expert	Password `Nestself-dev-1234!`, then authenticator enrolment

For TOTP in development use any authenticator app with the QR code, or compute codes with `computeTotp(secret)` from `apps/api/src/identity/totp.ts`. The web apps run in sample mode by default (`VITE_DATA_SOURCE=sample`); set `VITE_DATA_SOURCE=api` in `apps/workspace/.env.local` or `apps/admin/.env.local` to use the real sign-in screens against the API. Set `SESSION_COOKIE_INSECURE_DEV=true` in the API env so the session cookie works on plain `http://localhost`.

Switching to a real Cognito pool is described in section 17.

3. Repository map

Path	What lives there
`apps/api/src/auth/`	HTTP sign-in layer: `auth.controller.ts`, `authentication.guard.ts`, the sign-in services, challenge store, throttle, bearer verifiers, `access-revocation.service.ts`, `profile.service.ts`
`apps/api/src/authz/`	Authorization: `authorize.ts` engine and helpers, `authorization.guard.ts`, `authorize.decorator.ts`, `public.decorator.ts`, `authenticated.decorator.ts`, `resolvers.ts`, `context-loader.ts`, golden decision table, `default-deny.test.ts`
`apps/api/src/principal/`	`principal-loader.ts` (user → `Principal` data with Redis caching), `cache-gateway.ts` (circuit breaker), `invalidation-subscriber.ts`
`apps/api/src/session/`	Redis sessions: `session.service.ts`, `session-cookie.ts`, `csrf.guard.ts`, `session-policy.ts` (lifetimes), `token-cipher.ts`
`apps/api/src/identity/`	`identity-provider.service.ts` (abstract contract), `cognito-identity-provider.ts` + helpers, `fake-identity-provider.ts` + `fake-user-store.ts`, `totp.ts`
`apps/api/src/redis/`	`KeyValueStore` contract, Redis and in-memory implementations, `redis-keys.ts` (every key and TTL)

Path	What lives there
`apps/api/src/db/`	Drizzle: `db.service.ts`, `schema/*.ts`, `repositories/*.ts`, `seed/`, `migrate.ts`
`apps/api/drizzle/`	SQL migrations and drizzle-kit snapshots
`apps/api/src/audit/`	`audit-writer.ts`, `audit-events.ts` (action catalogue and categories), `pino-redaction.ts`
`apps/api/src/invitations/`, `members/`, `org-units/`, `users/`	Organization and operations administration endpoints
`apps/api/src/ws/`	Socket.IO adapter, gateway, handshake authentication, invalidation
`apps/api/src/jobs/`	`job-authorization.ts` wrapper for BullMQ processors
`apps/api/src/common/`	`api-error.ts` (error envelope), exception filter, request metadata, Zod pipe
`apps/api/src/config/env.ts`	Zod schema of every environment variable, `AUTH_MODE` rules
`apps/api/test/`	Integration suites against a real PostgreSQL (`*.e2e.spec.ts`, `db/schema.spec.ts`)
`apps/api/docs/auth-core.md`	Route table and module map kept current with the code
`packages/contracts/src/`	Shared Zod schemas and types: `auth.ts`, `principal.ts`, `authorization.ts`, `actions.ts`, `templates.ts`, `org-admin.ts`
`packages/api-client/src/auth.ts`	Typed client for every auth endpoint (cookie or bearer mode)
`apps/workspace/src/auth/`, `apps/admin/src/auth/`	`AuthProvider`, sign-in screens, error mapping
`apps/mobile/src/auth/`	Cognito public client (PKCE, email OTP), secure token store, `AuthProvider`
`infra/cdk/lib/auth-stack.ts`, `infra/cdk/lib/triggers/`	Cognito user pool, app clients, Lambda triggers, secrets
`packages/sample-data/src/fixtures/`	Fictional people, org units, credentials, `identities.ts` (fake pool subjects)

4. The request pipeline

Four global guards run in this order for every HTTP request (`apps/api/src/app.module.ts`):

1. **`AuthenticationGuard`** (`src/auth/authentication.guard.ts`). Skips `@Public()` routes. Otherwise it reads the session cookie or the `Authorization: Bearer` header, resolves a `Principal` and attaches `request.principal`, `request.session` (web only), `request.authMeta` and `request.csrfToken`. Missing or invalid credentials end the request with 401.
2. **`CsrfGuard`** (`src/session/csrf.guard.ts`). Only when a session cookie is present and the method is not safe: it checks `Sec-Fetch-Site`, the `Origin` header against the allowed surfaces and the double-submit token (`X-CSRF-Token` header must equal the CSRF cookie). WebSocket contexts are skipped. Failure is 403 `csrf_rejected`.

3. **`AuthorizationGuard`** (`src/authz/authorization.guard.ts`). Skips `@Public()` and `@Authenticated()` routes. For `@Authorize(action, resolver)` it calls the resolver to derive the `Resource`, builds the context with `ContextLoader`, calls `authorize()`, writes an `access.denied` audit event on denial (the engine's reason is recorded there, never returned to the client) and answers 404 when the caller has no visibility into the organization and 403 otherwise (AUTHZ-R-61). A handler that carries no decorator at all is denied at runtime as well as by the CI test.
4. **`ThrottlerGuard`** (NestJS throttler, 60 requests per minute by default; sign-in routes carry tighter `@Throttle` values).

Decorators (`src/authz/`):

Decorator	Meaning
`@Public()`	No authentication. Sign-in, invitation preview and accept, health, leads.
`@Authenticated()`	A valid session or bearer token, no action check. `/auth/me`, sessions, sign-out, context switch, profile.
`@Authorize(ACTIONS['x.y'], resolver)`	Full grant-based check. The resolver returns `{ type, id?, orgId?, ... }`; use `orgResourceFromContext(type)` for `/orgs/:orgId/*` routes (it enforces that the active context is that organization, API-R-21) and `operationsResource(type, idFn)` for `/ops/*` routes (session required, operations context).
`@CurrentPrincipal()`	Parameter decorator returning `request.principal`.

`src/authz/default-deny.test.ts` boots the application and fails when any controller handler or gateway message handler lacks one of the three decorators. It runs with the normal test suite, so an undecorated route cannot reach `main`.

Errors use one envelope (`src/common/api-error.ts`, `api-exception.filter.ts`): `{ status, code, message, requestId, details? }`. Codes the clients map: `unauthenticated`, `session_expired`, `csrf_rejected`, `forbidden`, `not_found`, `context_mismatch`, `session_required`, `invalid_code`, `invalid_credentials`, `invalid_challenge`, `too_many_attempts`, `validation`, `duplicate_invitation`, `already_member`.

5. Identity provider layer

`IdentityProviderService` (`src/identity/identity-provider.service.ts`) is an abstract class used as the DI token. It covers everything the API needs from the identity provider: start and answer the email OTP challenge, password sign-in and the `SOFTWARE_TOKEN_MFA`, `NEW_PASSWORD_REQUIRED` and `MFA_SETUP` challenges, TOTP association and verification, refresh and revocation, global sign-out, enable, disable, create, delete and find users, forgot-password. Sign-in services depend only on this contract, so swapping the provider never touches them.

- **`CognitoIdentityProvider`** (`AUTH_MODE=cognito`): AWS SDK v3 calls with the confidential web client (`SECRET_HASH` on every call). User-management calls live in `cognito-user-ops.ts`; `cognito-error-map.ts` turns Cognito exceptions into `IdentityProviderError` codes.
- **`FakeIdentityProvider`** (`AUTH_MODE=stub`): in-memory users, sessions, OTP codes and refresh-token families with the same challenge chain as Cognito (`fake-user-store.ts`). It refuses to construct when `NODE_ENV=production`, logs email codes, honours `FAKE_IDP_FIXED_OTP`, and is pre-seeded with the sample people in development.

Both return `AuthOutcome`, either `{ kind: 'tokens', tokens }` or `{ kind: 'challenge', challenge, session, username }`.

6. Sign-in flows

All flows start at `POST /auth/discover` (`discover.service.ts`), which answers `email_otp`, `password` or `sso` without revealing whether the address exists. Interactive steps are stitched together with **challenge tokens**: `ChallengeStore` (`src/auth/challenge-store.ts`) stores the provider session under `challenge:{sha256(token)}` for 5 minutes and consumes it on first use. Failed attempts count in `SignInThrottle` (`throttle:otp:{email}`): five failures lock the identifier for 15 minutes with 429 `too_many_attempts`. Unknown addresses go through the same timing (`neutral-delay.ts`) and receive a dummy challenge so responses do not reveal existence.

Employees: email code. `POST /auth/email-otp/start` → `{ challengeToken }`; `POST /auth/email-otp/verify` with `{ challengeToken, code }` → tokens → `IdentityResolutionService.resolveIdentityAndCreateSession()` maps the token `sub` to an `external_identities` row, loads the principal, creates the Redis session, sets the cookies and returns `{ principal, effectiveActions }`.

Experts and staff: password and authenticator. `POST /auth/password` returns either the signed-in principal or `{ step, challengeToken }` where `step` is `new_password`, `totp` or `totp_enrol`. `POST /auth/challenge` with `{ challengeToken, response }` answers one step and returns either the next `{ step, challengeToken }` (200) or the principal. For `totp_enrol` the client first calls `POST /auth/totp/enrol` with the challenge token to receive the `otpauth://` URI and a rotated challenge token, then `POST /auth/challenge` with the authenticator code; the API verifies the software token and completes `MFA_SETUP` in one step (`sign-in.service.ts`, `totp.service.ts`). `POST /auth/totp/confirm` is the signed-in variant for adding an authenticator later.

Password reset. `POST /auth/password-reset/start` and `/confirm` (`password-reset.service.ts`) wrap the provider's forgot-password flow with neutral responses.

Invitation acceptance. `GET /invitations/:token/preview` returns only the organization name; `POST /invitations/accept` (`src/invitations/invitation-activation.service.ts`) validates the hashed token, creates or links the user, ensures the identity-provider user and the `external_identities` row, creates the ACTIVE membership and the template grants in one transaction, then starts the right sign-in method: an email code for employees or a temporary-password flow for staff and experts.

SSO. `GET /auth/sso/start` and `/callback` are not implemented yet (phase 3); the discover step still reports `sso` when a verified domain matches an active connection.

Every sign-in writes `auth.sign_in.succeeded` or `auth.sign_in.failed` through `AuditWriter`.

7. Sessions, cookies and CSRF

`SessionService` (`src/session/session.service.ts`) stores one JSON record per session at `session:{sha256(sessionId)}` and indexes a user's sessions in `user_sessions:{userId}`. The record (`session.types.ts`) holds the user id, surface (`workspace` or `admin`), population, active context, authentication metadata, lifetimes, a hashed IP and user-agent family, the CSRF token, and the Cognito

tokens encrypted with AES-256-GCM (`token-cipher.ts`, key `SESSION_ENCRYPTION_KEY`, dual-key rotation via `SESSION_ENCRYPTION_KEY_PREVIOUS`).

Cookies (`session-cookie.ts`): `__Host-nestself_session` and `__Host-nestself_csrf`, `Secure`, `HttpOnly` for the session, `SameSite=Lax`, `Path=/`. `SESSION_COOKIE_INSECURE_DEV=true` drops the `__Host-` prefix and `Secure` for plain-HTTP localhost. The CSRF cookie is readable by JavaScript on purpose: the API client copies it into `X-CSRF-Token`.

Lifetimes (`session-policy.ts`) depend on population and context, for example employee personal 30 minutes idle and 12 hours absolute, employee organization 60 minutes and 8 hours, expert 8 hours and 7 days, staff 60 minutes and 8 hours; `SESSION_LIFETIME_OVERRIDES` (JSON) adjusts them per environment within the 30-day idle and 90-day absolute caps. Each request extends the idle window; the absolute expiry never moves. The session id is regenerated at sign-in and will be again after step-up.

Access-token refresh happens lazily: `ensureFreshAccessToken` refreshes through the identity provider under `lock:session:{hash}` so concurrent requests do not race; a failed refresh destroys the session.

Revocation paths: `POST /auth/sign-out` (current session), `POST /auth/sign-out-all` (every session, Cognito global sign-out), `DELETE /auth/sessions/:id` (another device), and `AccessRevocationService.revokeAllAccess()` (`src/auth/access-revocation.service.ts`), which membership end and suspension call: destroy sessions, `globalSignOut` for every external identity, publish the principal invalidation. It never throws; it reports what failed.

Context switch (`POST /auth/context`, `session-endpoints.service.ts`) changes the active context (`personal`, `organization`, `expert`, `operations`) after checking the principal actually holds it; it never extends the absolute expiry.

8. Mobile bearer tokens

Mobile apps authenticate against Cognito directly and send `Authorization: Bearer <access token>`. `AuthenticationGuard` passes the token to the `BEARER_VERIFIER` provider (`src/auth/bearer-verifier.ts`): `CognitoBearerVerifier` verifies signature and claims with `aws-jwt-verify` (`tokenUse: 'access'`, the mobile client id) and rejects tokens whose refresh family is listed in `revoked_family:{originJti}`; `StubBearerVerifier` accepts HS256 tokens signed with `JWT_SECRET` in stub mode. The guard then maps `sub` to the user through `external_identities`, records the family in `user_mobile_families:{userId}` and refreshes `mobile_family:{originJti}` (30-day TTL, AUTHZ-R-64) and assembles the principal with the default context for the user. Bearer callers cannot use `/ops/*` routes (API-R-28) and get 403 `session_required`. The nightly `mobile.prune-stale-families` job (section 14) denylists families whose last-seen key has expired.

Cognito access tokens carry `auth_time` but not the method, so `CognitoBearerVerifier` reports `email_otp` with no second factor. That is accurate today: the mobile app client only permits the email-code first factor (`USER_AUTH` with `EMAIL_OTP`), passwords and authenticators are web-only, and SSO (which would set `sso`) is phase 3. Step-up for mobile goes through the application-side step-up endpoints (phase 2), never through claims.

9. Principal, grants and the authorization engine

Shape. `Principal` (`packages/contracts/src/auth.ts`, `principal.ts`): `userId`, `displayName`, `initials`, `locale`, `memberships[]` (id, orgId, orgUnitId, status, dates), `staffTemplates[]`, `expert` (id and approved categories), `context`, `auth` (method, time, second factor, last step-up), `sessionId`, `tokenFamily`. It carries no grants; those are loaded per decision.

Loading and caching. `PrincipalLoader` (`src/principal/principal-loader.ts`) reads through the repositories and caches in Redis with jittered TTLs: `principal:{userId}` 30 s, `grants:{userId}:{orgId}` 60 s, `assignments:{userId}` 60 s, `org_tree:{orgId}` 5 min. `CacheGateway` wraps Redis with a circuit breaker (three failures open it for 5 s) and a small in-memory LRU fallback. Writes that change access call `invalidateUser(userId, orgIds)`, which deletes the keys and publishes on the `invalidate` channel; `InvalidationSubscriber` on every API instance drops its local copies, and the WebSocket layer re-authorizes connected sockets.

Vocabulary. `ACTIONS` (`packages/contracts/src/actions.ts`) is the closed catalogue of dotted action strings (`people.invite`, `program.overview`, `conversation.read.assigned` …). `PERMISSION_TEMPLATES` (`templates.ts`) are the 17 named starting sets (`platform_owner`, `client_operations`, `hr_admin`, `team_manager`, `employee`, `expert` …) with default actions, scope type, conditions and step-up actions. A **grant** gives a user a list of actions within a scope (`own`, `org_unit` with optional descendants, `organization`, `platform`, `assigned_cases` …) with optional Zod-validated conditions (spending ceiling, categories, confidential approver, time window), a validity window and, for delegated grants, a parent and depth limit of 3.

Deciding. `authorize(principal, action, resource, ctx)` (`src/authz/authorize.ts`) is pure and ordered: user state (suspended or deactivated denies everything), context validity (`context.ts`), confidential hard deny, assignments, grants with scope matching through the closure-table ancestor set (`scope.ts`), conditions (`conditions.ts`), validity window, self-approval rule, delegation envelope (`delegation.ts`). It returns `{ allowed, grantIds, assignmentIds, denialReason, explanation[] }`; `explain.ts`, `authorize-many.ts` and `effective-access.ts` build on it. `ContextLoader` (`context-loader.ts`) assembles the `AuthorizationContext` (grants, assignments, ancestor ids, organization settings, emergency access for confidential actions, which is never cached). The golden table `golden-table.json` (140 tuples) and fast-check property tests (`authorize.property.test.ts`) pin the behaviour: no cross-organization allow, delegation never exceeds the delegator, expired never allows, confidential never allowed by administrative scopes.

Using it outside a guard. `AuthorizationService.require(principal, action, resource)` for service-layer checks, `effectiveActions(principal)` for `/auth/me`. Clients get the same answer client-side as a hint through `can()` in `packages/contracts/src/principal.ts`; it is never the enforcement.

Step-up. `STEP_UP_CLASS_BY_ACTION` marks actions that need recent re-authentication; the guard annotates `request.stepUpClass`. Enforcement and the `/auth/step-up/*` endpoints are phase 2.

10. Data layer: Drizzle, repositories and row-level security

Schema (`src/db/schema/`): `identity.ts` (users, external_identities), `organization.ts` (organizations, org_units, org_unit_closure, memberships, legacy_teams and the `teams` view), `authorization.ts` (permission_templates, grants, assignments, approval_authorities, emergency_access), `invitations.ts` (invitations, sso_connections, sso_domains), `audit.ts` (audit_events, retention_policies), `content.ts` (experts, credentials, content, channels, polls, leads, event requests), `relations.ts` for `db.query.*`.

Conventions: snake_case columns (`casing: 'snake_case'`), text ids with `gen_random_uuid()::text` defaults, `timestamptz(3)`, `updated_at` maintained by Drizzle.

Row-level security. Tables with an organization column (`org_units`, `memberships`, `grants`, `approval_authorities`, `invitations`, `sso_connections`, `channels`, `event_requests`) declare a `pgPolicy` whose predicate is `coalesce(current_setting('app.current_org_id', true), '') = '' OR org_id = current_setting('app.current_org_id', true)` (platform-wide grants with a null organization stay visible). Tables that reach their organization only through a join use `tenantPredicateVia()` with a correlated `EXISTS`: `posts` through `channels`, `polls`, `poll_options` and `poll_votes` through `legacy_teams`, and `org_unit_closure` through `org_units` (migration `0002_rls_legacy_content_closure.sql`). Every one of the thirteen tenant tables has RLS enabled and forced. `DbService.withOrgScope(orgId, fn)` runs `fn` in a transaction after `select set_config('app.current_org_id', $1, true)`; everything inside sees and writes only that organization. Code that runs without the setting (principal loading, seeds, jobs) is in system context and sees all rows. Policies bind only roles that are neither superuser nor table owner: migrations run as the owner, the API connects as a login that is a member of `nestself_app` (Docker: `nestself_api` created by `docker/initdb/01-app-role.sql`). A local Homebrew superuser is not restricted; `test/db/schema.spec.ts` proves the policies with a non-superuser login.

Repositories (`src/db/repositories/`): `identity`, `authorization`, `audit`, `leads`, `invitations`, `memberships`, `org-units`. They are the only place with Drizzle query builders; every method accepts an optional `tx` so services can compose them inside `withOrgScope`. Services depend on repositories, which is why unit tests mock plain methods and app-level tests override `DbService` with `{ ping, onModuleInit, onModuleDestroy }`.

Migrations. `pnpm --filter @nestself/api db:generate` diffs the schema against `drizzle/meta` and writes a new SQL file; `db:generate -- --custom --name=<name>` creates an empty file for hand-written SQL; `db:migrate` applies pending files (tracked in `drizzle.__drizzle_migrations`); `db:seed` upserts the fixtures. `0001_audit_partitions_rls_roles.sql` holds what drizzle-kit cannot express: monthly range partitions and `audit_events_ensure_partitions()`, the append-only triggers, the `nestself_app` role and its privileges, `FORCE ROW LEVEL SECURITY`. `0002_rls_legacy_content_closure.sql` adds the join-based policies above, forces them, and makes `audit_events_ensure_partitions()` `SECURITY DEFINER` so the application role can create partitions from the nightly job. Never let drizzle-kit alter `audit_events` directly; write a custom migration.

11. Organization and operations administration

All `/orgs/:orgId/*` routes require the active context to be that organization (`orgResourceFromContext`), run their writes inside `withOrgScope`, and audit through `AuditWriter`.

Area	Routes	Files
Invitations	`POST /orgs/:orgId/invitations`, `POST .../import`, `DELETE .../:invitationId`; public `GET /invitations/:token/preview`, `POST /invitations/accept`	`src/invitations/` (`invitations.service.ts`, `invitation-activation.service.ts`, `invitation-token.ts`, `invitation-mailer.ts`, `ses-invitation-mailer.ts`)
Org units	`GET`, `POST /orgs/:orgId/org-units`, `PATCH .../:unitId`, `POST .../:unitId/move`	`src/org-units/` (`closure-moves.ts` holds the pure closure-table algebra)

Area	Routes	Files
Members	`GET /orgs/:orgId/members` (cursor pagination, filters), `PATCH .../:membershipId`, `POST .../:membershipId/end`	`src/members/`
Operations	`POST /ops/users/:userId/suspend`, `POST .../reinstate`	`src/users/`

Invitation tokens are 32 random bytes; only the SHA-256 hash is stored, with a 7-day expiry. In development the create response adds `devToken` and the logging mailer prints the link; with `SES_FROM_EMAIL` set the SES mailer sends it. Membership end sets `ENDED`, revokes the user's grants scoped to the organization and their delegated children (depth 3), audits first, then calls `revokeAllAccess` (SESS-R-22, SESS-R-23). Suspension sets `users.suspended_at`, disables the identity-provider user, audits `user.suspended` (SECURITY tier) and revokes all access; `authorize()` denies suspended users on the next request anyway.

`people.deactivate` is held by three templates: `hr_admin` (organization scope, ends memberships of its own organization through `/orgs/:orgId/members/:id/end`), `platform_owner` (platform scope, the only template that reaches `/ops/users/:id/suspend`, whose resource has no organization) and `client_operations` (scope `assigned_clients`, for offboarding on behalf of an assigned client). The third is declared but not yet reachable over HTTP: `/orgs/:orgId/*` routes require the organization context (API-R-21) and staff hold no membership, so phase 2 adds `/ops/orgs/:orgId/...` routes whose resolver passes the route organization into the resource and lets the engine check the CLIENT assignment. The sample data has no platform owner; tests create that grant from the template.

12. Audit log

`AuditWriter.record()` (`src/audit/audit-writer.ts`) inserts into `audit_events` with the category derived from the action (`audit-events.ts`: `auth.*` and `user.suspended` are SECURITY, `grant.*` and `access.denied` AUTHORIZATION, administration ADMIN, content CONTENT_ACCESS), the retention horizon from `retention_policies` (10-minute cache, defaults per tier), a keyed hash of the IP (HMAC with `IP_HASH_SALT`, falling back to the session encryption key, so it cannot be reversed from a table of all addresses) and the parsed user-agent family. Details are sanitised: forbidden keys (tokens, passwords, message bodies, intake text, notes, voucher codes, raw email) are dropped and strings truncated. `record()` never throws in production; tests set `strictMode`. The table is range-partitioned by month and protected by triggers that reject UPDATE and DELETE; the application role also lacks those privileges. `pino-redaction.ts` redacts the same fields from HTTP logs. Add a new action to `AUDIT_ACTIONS` before using it; the type will not compile otherwise.

13. WebSockets

`NestSelfIoAdapter` (`src/ws/nestself-io.adapter.ts`, installed in `main.ts`) serves Socket.IO under `/api/ws`. The handshake middleware (`ws-auth.service.ts`) checks the `Origin` against the allowed surfaces, then authenticates either the web session cookie (same code path as HTTP) or a mobile ticket from `POST /auth/ws-ticket`: 30-second TTL, hashed, consumed atomically with `GETDEL` (`ws_ticket:{hash}`). The gateway (`events.gateway.ts`) exposes `subscribe { room, action, resource }` and `unsubscribe`; each subscription runs `authorize()` (`subscription-authorizer.ts`) and denied rooms emit `subscribe:denied`. `ws-invalidation.service.ts` listens on the `invalidate` channel and, for connected

sockets of that user, reloads the principal, disconnects suspended or deactivated users and leaves rooms whose grants are gone (`subscription:revoked`). `ws-connection-registry.ts` tracks sockets per user.

14. Background jobs

Two kinds of job live in `src/jobs/` .

Jobs on behalf of a principal. `JobAuthorization` (`job-authorization.ts`) wraps BullMQ processors. A job payload carries `{ principalId, context, action, resource, data }` and never tokens or grants (AUTHZ-R-54). At execution the wrapper reloads the principal, rebuilds the context and calls `authorize()` ; an unknown, suspended or denied principal completes the job as `skipped` (not failed, so no retry) with an `access.denied` audit event, while allowed jobs run the handler and audit with `actorKind: 'JOB'` . Feature modules that enqueue work (exports, imports) wrap their processors with it.

System maintenance jobs. `MaintenanceJobsService` (`maintenance-jobs.service.ts`) holds three idempotent jobs that run in system context (no tenant setting) and audit with `actorKind: 'JOB'` , `actorUserId: null` :

Job	Schedule (local)	What it does
`audit.ensure-partitions`	01:00	Calls `audit_events_ensure_partitions(3)` so the next three months' partitions exist (D-21)
`grants.reconcile`	02:15	Closes grants whose `valid_until` passed (`grant.expired`), revokes delegations whose parent is revoked (`grant.delegation_revoked`), revokes organization-scoped grants whose subject has no ACTIVE membership (`grant.revoked` , reason `membership_ended`), then invalidates each affected principal (T-2.10, D-17)
`mobile.prune-stale-families`	02:30	Scans `user_mobile_families:*` ; a family whose `mobile_family:{jti}` key has expired (30 days unseen) is denylisted in `revoked_family:{jti}` for 90 days, removed from the set and audited as `auth.mobile_family.revoked` (AUTHZ-R-64)

`JobSchedulerService` (`job-scheduler.service.ts`) registers these as BullMQ job schedulers on the `nestself-maintenance` queue (its own ioredis connection to `REDIS_URL` , cron in `JOBS_TIMEZONE` , default `Europe/Belgrade` , three attempts with exponential backoff) and runs a worker with concurrency 1. It is started only by the worker process: `src/worker/main.ts` boots the application context without an HTTP listener and calls `start()` ; the ECS `worker` service runs `node dist/worker/main.js` from the same image as the API, locally `pnpm --filter @nestself/api worker:dev` . Without `REDIS_URL` the worker exits with an error. `pnpm --filter @nestself/api job:run <name>` executes one job immediately through `scripts/run-job.ts` and prints its report, which is how to verify a job after a deploy or catch up after downtime. `KeyValueStore.scan()` exists for these jobs only; request paths never call it.

Unit tests (`maintenance-jobs.service.test.ts`) use mocked repositories and the in-memory store; `test/jobs.e2e.spec.ts` runs the reconciliation and partition jobs against the real database and checks the audit trail and the application role's permission to create partitions.

15. Web clients

`packages/api-client/src/client.ts` works in cookie mode (default): `credentials: 'include'` and the CSRF cookie copied into `X-CSRF-Token` on mutating requests. `auth.ts` exposes typed functions for every endpoint (`discover`, `emailOtpStart`, `emailOtpVerify`, `password`, `challenge`, `totpEnrol`, `me`, `switchContext`, `signOut`, `listSessions`, `invitationAccept` ...) validated with the contracts schemas.

`apps/workspace/src/auth/` and `apps/admin/src/auth/`: `AuthProvider` calls `GET /auth/me` on mount and window focus and exposes `status`, `principal` and `effectiveActions` through `useAuth()`; a 401 flips to unauthenticated. `sample-auth-provider.tsx` serves sample mode with `SAMPLE_PRINCIPALS`. `screens/sign-in-page.tsx` drives the identifier-first flow with one screen per step (`identifier`, `otp`, `password`, `new-password`, `totp`, `totp-enrol`); `map-auth-error.ts` maps error codes to localized messages (`packages/i18n/locales/*/auth.json`). Routes under `routes/_authenticated/` redirect to `/sign-in` without a principal and gate sections with `can()` (for example `_hr.tsx` requires `program.overview`); `routes/_public/invite.$token.tsx` handles invitation links.

16. Mobile client

`apps/mobile/src/auth/`: `cognito-public-client.ts` talks to Cognito directly (`InitiateAuth` with `USER_AUTH` for email codes, refresh, revoke), `sso.ts` prepares the PKCE authorization-code flow through `expo-auth-session`, `token-store.ts` keeps tokens in `expo-secure-store` (device-only, unlocked access), `jwt.ts` decodes claims, `auth-state.ts` is the reducer and `auth-provider.tsx` orchestrates sign-in, refresh, sign-out and the sample mode. API calls use the bearer mode of the shared client; WebSocket connections fetch a ticket first. Mobile settings come from `EXPO_PUBLIC_COGNITO_*`.

17. Infrastructure and the Cognito rollout

`infra/cdk/lib/auth-stack.ts` defines the user pool `nestself-<stage>` (Essentials plan, email OTP and password first factors, optional TOTP, no self sign-up), three app clients (confidential web with secret, public mobile with PKCE, reserved machine-to-machine), the `CustomMessage` and `PreSignUp` Lambda triggers (`lib/triggers/`), the web client secret and the session encryption key in Secrets Manager, and optional WAF, SES and custom domain for staging and production. Outputs: `UserPoolId`, `WebClientId`, `MobileClientId`, `CognitoDomain`, `CognitoIssuer`, secret ARNs.

The CDK app contains only this stack unless `-c platform=true` is passed, which keeps the rollout to Cognito-only until the web applications ship (`infra/cdk/README.md`, "Rollout order"). To connect the API to a real pool:

```
pnpm --filter @nestself/cdk exec cdk bootstrap aws://<ACCOUNT_ID>/eu-central-1
pnpm --filter @nestself/cdk exec cdk deploy NestSelf-dev-Auth -c stage=dev -c account=
<ACCOUNT_ID>
node infra/cdk/scripts/env-from-outputs.mjs dev >> apps/api/.env     # COGNITO_* and the secret
```

The step-by-step account setup, credentials, IAM permissions and every environment variable are in `CDK-setup.md` at the repository root. Then set `AUTH_MODE=cognito`, `REDIS_URL` and `SESSION_ENCRYPTION_KEY` (from the stack secret) in `apps/api/.env`. Users must exist in the pool: create them through an invitation (the accept flow calls `AdminCreateUser`) or with the AWS console for a first test; the API links them by `sub`

on first sign-in only when an `external_identities` row exists, so onboarding goes through invitations. `env-from-outputs.mjs <stage> --mobile` prints the `EXPO_PUBLIC_*` values for the app (`EXPO_PUBLIC_COGNITO_CLIENT_ID` is the name `apps/mobile/src/auth/config.ts` reads).

The API process needs AWS credentials for the Cognito admin calls (`AdminInitiateAuth`, `AdminRespondToAuthChallenge`, `AdminCreateUser`, `AdminDeleteUser`, `AdminDisableUser`, `AdminEnableUser`, `AdminGetUser`, `AdminSetUserPassword`, `AdminUserGlobalSignOut`, `GetTokensFromRefreshToken`, `RevokeToken`, `ListUsers`, `DescribeUserPoolClient`) and, when `SES_FROM_EMAIL` is set, `ses:SendEmail`. In AWS these come from the ECS task role (`infra/cdk/lib/constructs/ecs-task-roles.ts`); locally from the default credential chain (`AWS_PROFILE`). The compute stack's `worker` service runs `node dist/worker/main.js` (section 14) with the worker task role.

18. Configuration reference

Validated in `apps/api/src/config/env.ts`; documented in `docs/environment.md`.

Variable	Purpose
`AUTH_MODE`	`stub` (default; refused in production) or `cognito`
`DATABASE_URL`, `DATABASE_POOL_MAX`	PostgreSQL connection for the API (a `nestself_app` member in Docker and AWS)
`REDIS_URL`	Required in cognito mode; optional in stub mode (in-memory store)
`JWT_SECRET`, `JWT_EXPIRES_IN`	Stub-mode bearer tokens only; removed after cutover
`COGNITO_REGION`, `COGNITO_USER_POOL_ID`, `COGNITO_WEB_CLIENT_ID`, `COGNITO_WEB_CLIENT_SECRET`, `COGNITO_MOBILE_CLIENT_ID`, `COGNITO_DOMAIN`, `COGNITO_ISSUER`	Pool coordinates; `env-from-outputs.mjs` prints them
`SESSION_ENCRYPTION_KEY`, `SESSION_ENCRYPTION_KEY_PREVIOUS`	32-byte hex keys for token encryption at rest; derived from `JWT_SECRET` in stub mode
`IP_HASH_SALT`	Key for the audit and session IP hashes; defaults to the session encryption key
`SESSION_COOKIE_NAME`, `CSRF_COOKIE_NAME`, `SESSION_COOKIE_INSECURE_DEV`	Cookie names and the localhost switch
`SESSION_LIFETIME_OVERRIDES`	JSON overrides per `population:context`
`MARKETING_URL`, `WORKSPACE_URL`, `ADMIN_URL`, `API_PUBLIC_URL`	Allowed origins for CORS, CSRF and WebSocket handshakes; invitation links
`TRUST_PROXY_HOPS`	Trusted proxy depth for client IP extraction
`SES_FROM_EMAIL`	Enables the SES invitation mailer

Variable	Purpose
`JOBS_TIMEZONE`	Worker process: IANA timezone of the nightly job schedules (default `Europe/Belgrade`)
`FAKE_IDP_FIXED_OTP`	Stub mode: an always-accepted six-digit code
`TEST_DATABASE_URL`	Integration tests; suites create `<name>_schema`, `<name>_auth`, ... databases

Redis keys and TTLs are listed in `apps/api/src/redis/redis-keys.ts`.

19. Testing

- Unit tests sit next to the code (`*.test.ts`, `*.spec.ts`) and mock repositories and services. The authorization engine has unit, property and golden-table tests.
- Integration suites in `apps/api/test/` boot the real application against a disposable PostgreSQL (`TEST_DATABASE_URL`, one database per suite via `test/db/test-database.ts`), use the fake identity provider and the in-memory store (the test environment ignores `.env` and `REDIS_URL`). `auth.e2e.spec.ts` covers sign-in, CSRF, context switch, profile, sessions and lockout; `invitations`, `members`, `org-admin` and `ws` cover their areas; `jobs.e2e.spec.ts` runs the maintenance jobs; `db/schema.spec.ts` proves migrations, partitions, append-only triggers and RLS on all thirteen tenant tables.
- Browser journeys for sign-in live in `e2e/auth/` and run with `pnpm e2e:auth` (`playwright.auth.config.ts`): they need the API in stub mode with `FAKE_IDP_FIXED_OTP=123456` and the web apps in `api` data mode; see `e2e/README.md`.
- `pnpm --filter @nestself/api test` runs everything; `pnpm test` runs the whole repository. CI (`.github/workflows/ci.yml`) provides Postgres and Redis service containers.

20. Recipes

Add a protected route. Put the handler in the right controller, pick the action from `ACTIONS`, and decorate with `@Authorize(ACTIONS['x.y'], orgResourceFromContext('thing'))` (or `operationsResource`). Read the body with `@Body(new ZodValidationPipe(Schema))`; schemas live in `packages/contracts/src/org-admin.ts`. Do writes inside `dbService.withOrgScope(orgId, tx => repo.method(..., tx))`, audit through `AuditWriter`, and invalidate caches that your change affects (`PrincipalLoader.invalidateUser`, or `org_tree:{orgId}` for tree changes). The default-deny test tells you if you forgot the decorator; add an e2e case to the matching suite.

Add an action or template. Extend `ACTIONS` in `packages/contracts/src/actions.ts` (and `CONFIDENTIAL_ACTIONS` or `STEP_UP_CLASS_BY_ACTION` where relevant), add it to the template in `templates.ts`, add golden-table tuples in `apps/api/src/authz/golden-table.json`, and re-run the seed so sample grants pick it up.

Add a table or column. Edit `apps/api/src/db/schema/*.ts`, run `db:generate`, review the SQL in `apps/api/drizzle/`, add a `pgPolicy` with `tenantPredicate()` if the table has an organization column, run

`db:migrate`, and extend the matching repository. Partitions, triggers and roles go in a `--custom` migration.

Add an audit action. Append to `AUDIT_ACTIONS` in `apps/api/src/audit/audit-events.ts` and map its category in `resolveCategory`; the spec file enumerates them.

Revoke someone's access programmatically. Write the audit event first, then `AccessRevocationService.revokeAllAccess(userId, { sessionsForOrgId?, orgIds? })`.

Debug a denial. Call `explain(principal, action, resource, ctx)` from `src/authz/explain.ts` in a test, or read the `access.denied` audit row: `details.reason` carries the engine's denial reason.

21. Known gaps and next steps

Phase 1 is complete; `docs/tasks/auth/phase-2-readiness.md` lists the phase 2 work in order with its preconditions. What remains open, and why:

- **Step-up re-authentication** is annotated (`req.stepUpClass`, `lastStepUpAt` in the session, `STEP_UP_MAX_AGE_SECONDS`) but not enforced. Enforcing it needs the `/auth/step-up/start|verify` endpoints first, otherwise administrators would be locked out of `people.deactivate` after 30 minutes. Phase 2, T-2.6.
- **Grant administration** (endpoints and screens), approval authorities, delegation, effective-access preview, emergency access and the confidential-domain tables with their RLS policies. Phase 2, T-2.1 to T-2.5.
- **Operations routes for assigned clients.** `client_operations` holds `people.import`, `people.onboard` and `people.deactivate` for assigned clients, but every administration route lives under `/orgs/:orgId/*`, which requires the organization context (API-R-21). Phase 2 adds `/ops/orgs/:orgId/...` routes with a resolver that passes the route organization into the resource.
- **Enterprise SSO** (phase 3): connection management, home-realm discovery and the callback are stubs.
- **Nightly jobs not yet written:** session TTL-drift reconciliation (SESS-R-16), the weekly Cognito user export (T-2.12), the retention sweeper (pending counsel on L-02 and L-03), the audit export to S3 with Object Lock (AUDIT-R-06) and the SSO certificate-expiry check (SSO-R-14). The scheduler and worker exist; each is a method on `MaintenanceJobsService` plus a schedule entry.
- **Compliance track** before the first real employee signs in: privacy notices, processing agreement, breach procedure, data-subject-rights endpoints (T-C.4).
- **Legacy community tables** (`posts`, `polls`, `poll_options`, `poll_votes`) and `legacy_teams` are now tenant-isolated but still deprecated; phase 2 retires them with the legacy team model (strategy Appendix A).
- **Mobile authentication strength** is reported as `email_otp` without a second factor because that is the only first factor the mobile client allows; it must be revisited when mobile gains passkeys or SSO.